

PROJECT AGENTIC_FACTORY_AI

Digital Factory Agentic AI

LangGraph 기반 RAG Agent

ML/DL 예지보전 모델을 Legacy System과 통합

2026.07.07

이은주

프로젝트 목차 (AGENDA)

01 프로젝트 여정 : ML → DL → LLM 연장선

02 문제 정의 및 프로젝트 목표

03 시스템 아키텍처

04 핵심 구현 요소 (7가지 경험 항목)

05 3개 Tool 상세 — 베어링 / 웨이퍼 / RAG

06 LangGraph 조건부 라우팅

07 LLM 백엔드 추상화 (API ↔ 로컬 sLLM)

08 데모 및 실행 방법

09 한계점 및 향후 개선 방향

10 결론 및 느낀점

프로젝트 여정 : ML에서 LLM으로

01

week1 ML 프로젝트 : 베어링 예지보전 시스템

- 🔧 RandomForestClassifier 기반 이진 분류 모델 (정상 vs 열화)
- 📊 XJTU-SY 공개 가속수명시험 데이터 학습 및 Streamlit Cloud 배포

h_rms (82%)

h_skew (12%)

h_crest (5%)

h_kurt (2%)

02

week2 DL 프로젝트 : 웨이퍼 결함 판별 시스템

- 🖼️ CNN 기반 이미지 분류 (128×128 웨이퍼맵 입력)
- 📁 LabelEncoder + Streamlit 기반 인터랙티브 웹앱 서비스 구축

Wafer Map Analysis

CNN Classification

Pattern Detection

03

week3 Digital Factory Agentic AI

- 🤖 LangGraph 기반 에이전트: 기존 ML/DL 모델을 하나의 Tool로 통합
- 🔍 하이브리드 RAG (Dense+Sparse) 결합으로 전문 지식 답변 강화

Agentic AI

LangGraph

Legacy Integration

데이터 생성



예측 & 패턴 분석



지능형 자동화 해석

문제 정의 : 왜 Agentic AI인가

기존 ML/DL 모델의 한계

모델의 개별성 (Siloed Models)

베어링 진단과 웨이퍼 판별 모델이 독립적으로 존재하여
통합 인터페이스 부재

사용 편의성 저하

피쳐값 직접 입력, .npy 배열 변환 등 비전문가가
사용하기 어려운 복잡한 입력 방식

해석의 기술적 장벽

추론 결과에 대한 맥락적 설명이 부족하여 전문가만이
최종 판단 가능

사용자 페인포인트

"베어링 이상 판정 결과가 나왔는데, 현장에서
그래서 어떻게 조치해야 하죠?"

"웨이퍼 결함 패턴이 감지되었는데, 이 패턴이
공정상 어떤 의미인지 설명이 필요해요."

"반도체 공정 가이드나 관련 기술 문서를 즉시
검색해서 답변해주는 AI는 없나요?"

해결 방향: Agentic AI

지능형 의도 파악 (Intent Recognition)

자연어 질문을 분석하여 베어링/웨이퍼/RAG 중 최적의
도구를 스스로 선택

통합 오케스트레이션

기존의 파편화된 ML/DL 자산을 단일 대화형
인터페이스로 완벽하게 통합

맥락적 답변 생성

단순 수치를 넘어 RAG를 활용한 도메인 지식 기반의 해결
방안 제시

프로젝트 목표 및 구현 범위



기존 모델 재활용

검증된 ML/DL 예지보전 모델을 Tool로 변환하여 코드 재사용성 극대화



자연어 질의 대응

복잡한 파라미터 입력 없이 대화만으로 설비 진단 및 반도체 Q&A 수행



환경 독립성 확보

Claude API와 사내 폐쇄망 로컬 sLLM 모두에서 동일한 로직으로 동작



7가지 핵심 구현 범위 (Experience Items)

RAG & DATA ENGINEERING



01. 문서 코퍼스 & 청킹

기술보고서 및 공정 문서의 오버랩 청킹을 통한 문맥 경계 손실 방지



02. 벡터 DB 구축

Multilingual-E5 임베딩과 Chroma DB를 활용한 지식 베이스 구축



03. 하이브리드 검색

Dense(의미) + Sparse(키워드) 검색을 결합한 EnsembleRetriever 구현

AGENTIC LOGIC



04. 모델 Tool 연동

베어링(RF) 및 웨이퍼(CNN) 모델을 LLM이 호출 가능한 Tool로 래핑



05. LangGraph 라우팅

State Machine 기반의 조건부 엣지를 설계하여 사용자 의도별 노드 분기

INFRASTRUCTURE & OPS



06. 백엔드 추상화

환경변수 설정을 통해 Claude API와 로컬 Ollama sLLM 간 즉시 전환



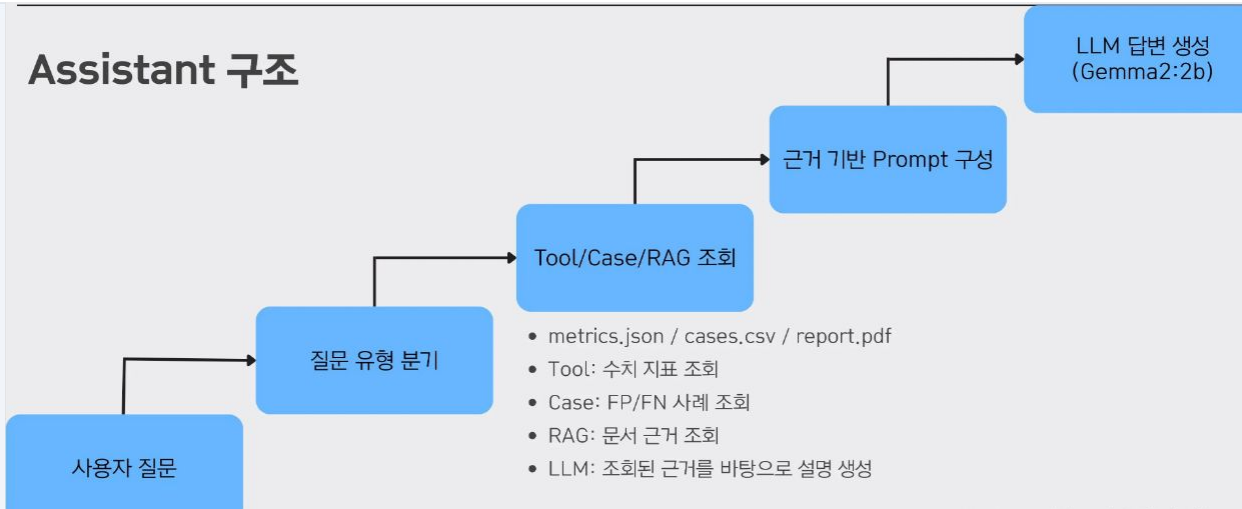
07. LangSmith 관찰성

전체 추론 과정 트레이싱 및 노드별 실행 흐름 시각화로 디버깅 최적화

전체 시스템 아키텍처

LangGraph Agentic Flow

Assistant 구조



지능형 라우팅 및 처리 프로세스

- 의도 분류 (Router Node)**
사용자 질문의 문맥을 분석하여 Bearing, Wafer, RAG, General 중 최적의 경로 선택
- 도구 및 지식 조회 (Tool/RAG)**
ML/DL 모델을 감싼 전용 Tool 호출 또는 하이브리드 검색을 통한 도메인 지식 추출
- 상태 기반 답변 생성**
추출된 근거(State)를 바탕으로 LLM이 최종 응답을 구성하여 사용자에게 전달

LangGraph State Machine

각 노드가 상태(State)를 공유하며 조건부 엣지로 유연하게 연결되는 구조입니다.

State Management
대화 이력, 검색 결과, Tool 출력을 통합 관리

Conditional Edges
질문 의도에 따른 실시간 분기 로직 구현

Bearing Route Wafer Route RAG Route General Route

핵심 파일 구조

graph.py
LangGraph 그래프 정의 및 라우팅 로직

tools.py
베어링/웨이퍼 모델 Tool 래퍼 구현

retriever.py
Dense + Sparse 하이브리드 검색 구성

config.py
LLM/임베딩 백엔드 추상화 설정

Tool 1 : 베어링 열화 감지



모델 아키텍처

알고리즘: RandomForestClassifier

모델 파일: bearing_rf_model.pkl

분류 목적: 진동 데이터를 통한 정상(0) 및 열화(1) 이진 판별

데이터 소스: XJTU-SY 공개 가속수명시험 진동 데이터
(2100rpm/12kN 조건)



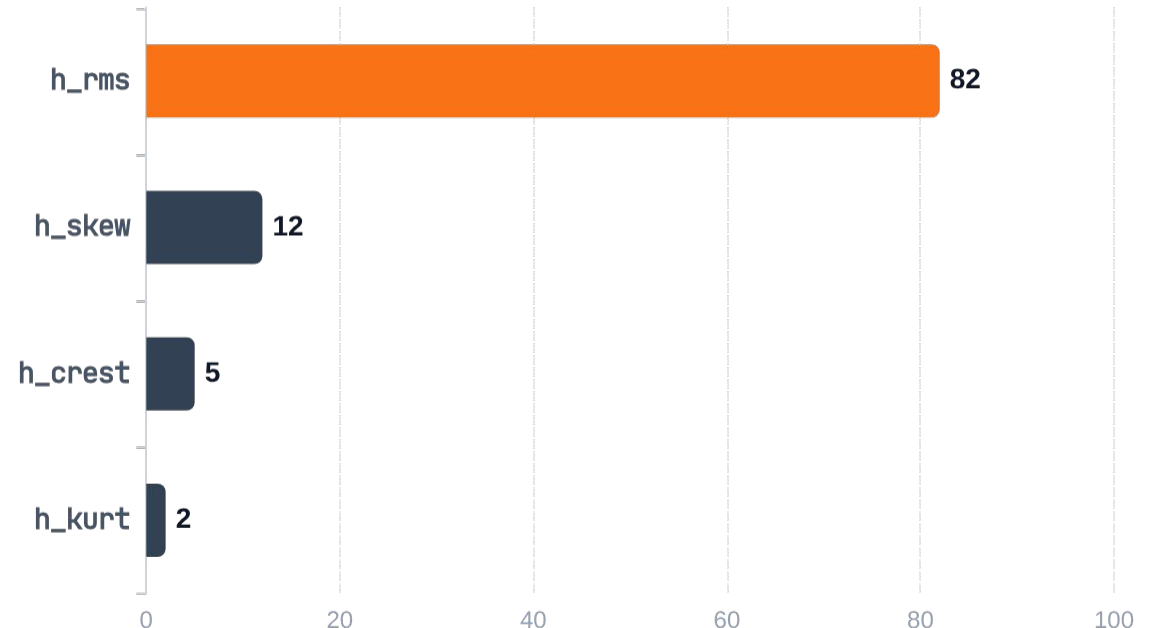
LLM 통합 메커니즘

도구명: predict_bearing_condition

동작 방식: 자연어 질문에서 4개 피처를 추출하여 추론 수행

결과 반환: 모델 예측값과 함께 해석된 상태 정보를 LLM에 전달

베어링 진단 피처 중요도 분석



* 각 피처의 모델 내 영향도 (Feature Importance) 비율 (%)

Tool 2 : 웨이퍼 결함 판별 (Wafer Defect Analysis)

 predict_wafer_defect

모델 아티팩트 및 규격

핵심 모델 : CNN (wafer_cnn_model.h5)

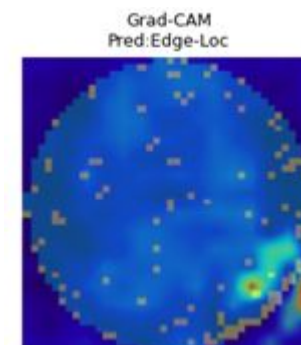
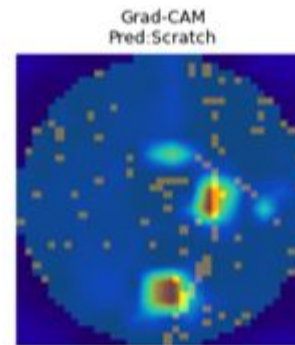
입력 형식 : .npy 2D Array (128×128)

데이터 규약 : 0(Outside), 1(Normal), 2(Defect)

레이블 복원 : LabelEncoder 기반 역변환 적용

 기존 DL 프로젝트 산출물(Model/Encoder)을
Agent Tool로 100% 재활용

웨이퍼맵 분석 및 결과 생성



[Visual] Grad-CAM Heatmap - 판정 근거 시각화 (XAI)



Data Load

NumPy 배열 전처리



CNN Inference

패턴 특징 추출/분류



NL Explanation

자연어 맥락 설명 생성

Tool 3 : 반도체 RAG 질의응답



지식 베이스 구성



CORPUS SOURCES

- **ML 기술보고서** : 베어링 예지보전 관련 자료
- **DL 발표자료** : 웨이퍼 결함 판별 시스템 문서
- **공개 반도체 문서** : WM-811K·XJTU-SY 원논문, SEMI E10·ISO 20816 표준 등 14종 (총 16문서 88청크)

DATA PROCESSING

문맥 경계 손실 방지를 위한 **오버랩 청킹 (Overlap Chunking)** 전략 적용



하이브리드 검색 엔진

ENSEMBLE RETRIEVER

Dense 검색과 Sparse 검색의 강점을 결합한 양상블 리트리버 시스템 구축

1. **Dense (Chroma)** : 의미론적 유사도 탐색
2. **Sparse (BM25)** : 정밀한 키워드 매칭
3. **Rerank/Ensemble** : 최종 최적 문서 선택

SEARCH GOAL

반도체 전문 용어에 대한 **키워드 정확도**와 질문 의도에 따른 **맥락 이해** 동시 확보



핵심 기술 사양

EMBEDDING MODEL

multilingual-e5-small

다국어 및 한국어 기술 문서 임베딩 최적화

VECTOR DATABASE

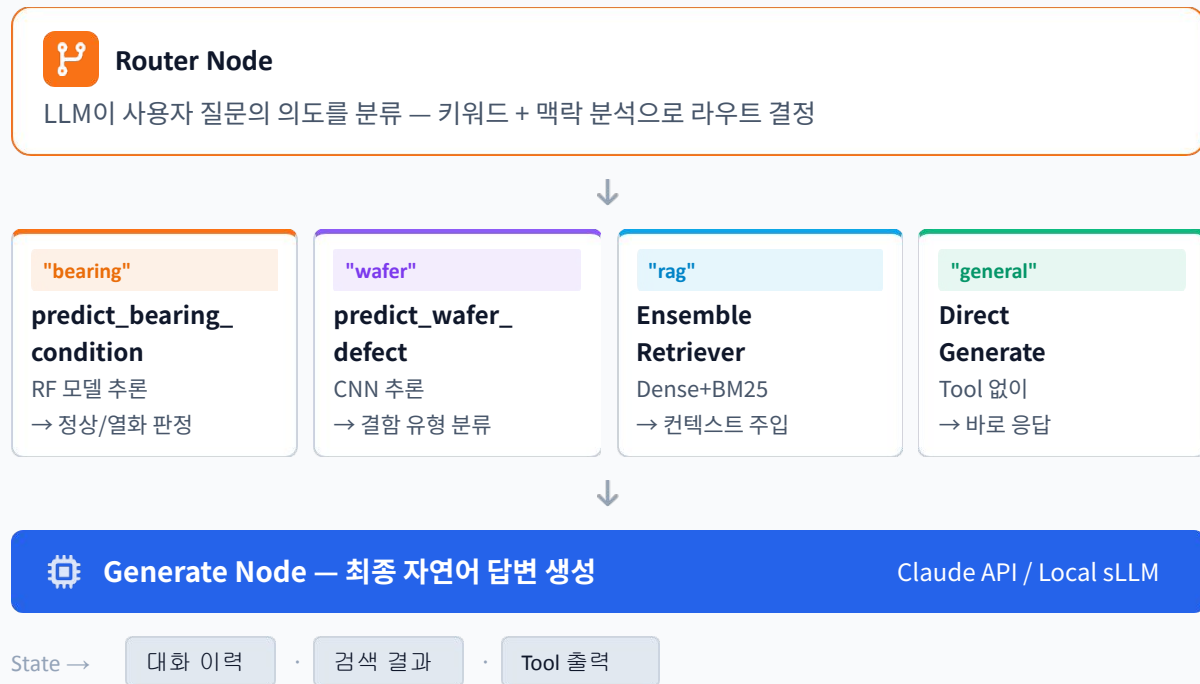
ChromaDB 기반의 고성능 벡터 저장 및 관리

ORCHESTRATION

LangGraph 내 **rag 라우트** 노드를 통해 질문 의도 발생 시 즉시 호출

LangGraph 조건부 라우팅

CONDITIONAL EDGE FLOW



LANGGRAPH 장점 & 구현



State 공유 — 노드 간 컨텍스트 유지

대화 이력, 검색 결과, Tool 출력을 State로 관리하여 모든 노드가 동일한 컨텍스트를 참조



재사용성 — 새 Tool 추가 용이

동일 그래프 구조에 신규 Tool(예: 온도·압력 센서) 추가 시 Edge 연결만으로 확장 가능



시각화 — 그래프 구조 다이어그램 확인

LangGraph 내장 시각화로 라우팅 경로와 노드 연결 관계를 직관적으로 파악 및 디버깅



라우팅 오류 디버깅 — LangSmith 연계

왜 rag 대신 general로 분류됐는지 노드별 실행 흐름을 트레이싱하여 정확한 원인 파악 가능

CORE FILE

`graph.py` → `build_graph` 함수 — LangGraph 그래프 정의 + 라우팅 로직

`tools.py` → 베어링/웨이퍼 Tool 래퍼 — 모델 추론 인터페이스

LLM 백엔드 추상화 : API와 로컬 sLLM



CLOUD BASE

API 모드 (Claude)

- ✓ Anthropic Claude Sonnet 4.6 활용
- ✓ 복잡한 추론 및 대규모 컨텍스트 처리에 최적화
- ✓ 인터넷 연결 필요, 높은 추론 속도 및 정확도



ON-PREMISE

로컬 모드 (Ollama)

- 🏠 Ollama 기반 Qwen2.5 (7B) 구동
- 🔒 사내 폐쇄망 및 보안 요구 환경 완벽 대응
- 🔧 외부 데이터 유출 차단, 인프라 비용 절감



설정 하나로 전환 (Zero-Code Change)

환경 변수 설정만으로 전체 시스템의 LLM 백엔드와 임베딩 모델을 즉시 교체합니다. 개발 환경(API)과 실 운영 환경(로컬)의 간극을 최소화합니다.

`LLM_BACKEND=local | api`



하이브리드 환경 지원

고성능 클라우드 모델을 통한 검증과 사내 보안 규정을 준수하는 로컬 실행을 모두 지원하여 프로젝트의 실용성과 배포 유연성을 확보했습니다.



일관된 인터페이스

LangChain/LangGraph의 표준 인터페이스를 활용하여, 백엔드가 바뀌더라도 Tool 호출 및 상태 관리 로직은 100% 재사용됩니다.

```
src/config.py — UNIFIED INTERFACE
def get_llm():
    backend = os.getenv("LLM_BACKEND", "api")

    if backend == "local":
        return ChatOllama(model="qwen2.5:7b-instruct")
    return ChatAnthropic(model="claude-sonnet-4-6")

# 애플리케이션 로직은 수정 없이 백엔드만 교체 가능
```

LangSmith 트레이싱 및 관찰



LLM 호출 및 토큰 추적

프롬프트 구성 및 모델 응답 전문 기록
실시간 토큰 사용량 및 API 호출 비용 모니터링
모델별 응답 지연 시간(Latency) 분석



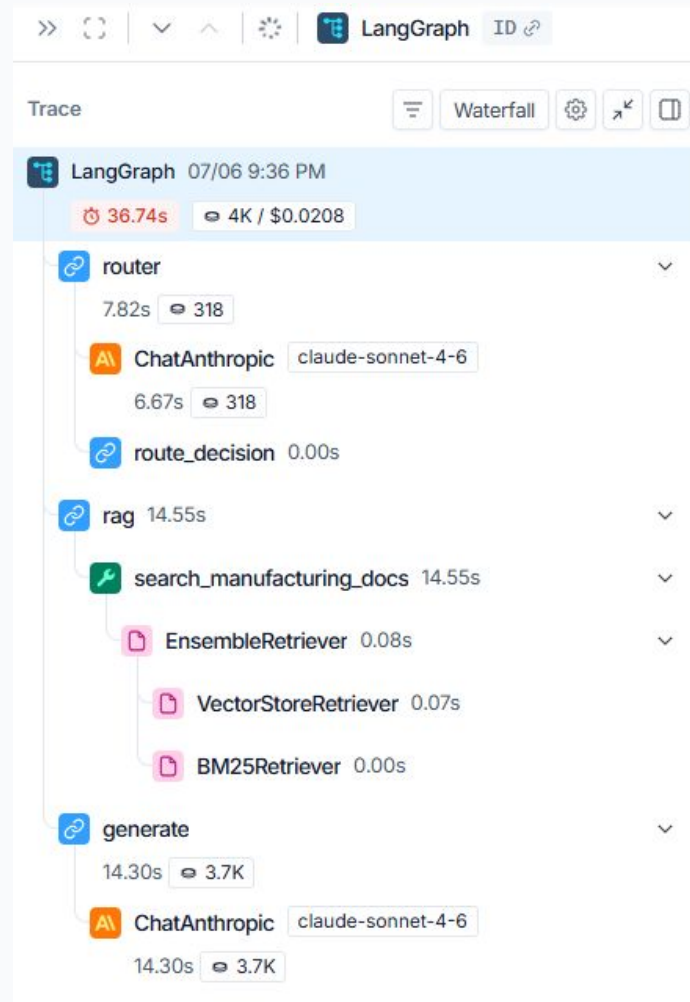
노드별 실행 흐름 시각화

LangGraph 상태 머신의 전체 실행 경로 추적
Node 간 데이터(State) 전달 과정 상세 확인
병렬 실행 및 조건부 분기 로직 검증



성능 지표 및 품질 평가

RAG 검색 결과의 유사도 및 적절성 평가
Tool 호출 성공률 및 파라미터 정확도 측정
사용자 피드백 기반 데이터셋 구축 및 테스트



데모 및 실행 방법

> 설치 및 실행 프로세스

- 1 의존성 설치: `pip install -r requirements.txt`
- 2 문서 배치: `corpus/` 폴더 내 `ml_`, `dl_` 접두사 파일 저장
- 3 환경 설정: API 키 및 `LLM_BACKEND` 변수 정의
- 4 앱 구동: `streamlit run src/app.py`

모델 아티팩트 연결

bearing_rf_model.pkl
베어링 열화 진단용 RF

wafer_cnn_model.h5
웨이퍼 결함 판별용 CNN

wafer_label_encoder.pkl
결함 유형 레이블 복원

chroma_db/
벡터 저장소 데이터베이스

단위 파이프라인 검증

✓ ingest_corpus.py

✓ retriever.py

✓ tools.py

✓ graph.py

* 개별 모듈 테스트를 통해 청킹, 검색, 도구 작동 및 그래프 구조를 독립적으로 확인 가능합니다.

주요 데모 시나리오

#BEARING TOOL

"h_rms=0.85, h_skew=0.3 등 수치 기반 베어링 상태는?"

#WAFER TOOL

"data/sample.npy 웨이퍼맵을 분석해서 결함 패턴 알려줘"

#RAG SEARCH

"Etch 장비 ESC O-ring이 빨리 열화되는데 어떤 소재로 교체하는 게 좋을까?"

알려진 제약사항 및 한계점



현재 시스템 제약 (Constraints)

- **개발 환경 이력**
초기 샌드박스 배선 검증 후 로컬 환경에서 실제 임베딩·모델·API로 전체 파이프라인 검증 완료
- **모델 아티팩트 의존성**
실제 ML/DL 모델(.pkl, .h5) 파일이 레거시 시스템에 종속되어 있어 Agent 배포 시 수동 배치 필요
- **데이터 입력 형식 제한**
웨이퍼 결함 분석 시 이미지 파일 직접 업로드가 아닌 .npy 배열 형식의 경로 전달 방식에 한정됨
- **정량 평가 체계 수립 중**
실제 문서 16개(자체 보고서 + 공개 논문·표준) 88청크 구축 완료, LangSmith 정량 평가 지표는 수립 단계



향후 개선 및 발전 방향 (Roadmap)

- **멀티모달 Tool 확장**
PIL 라이브러리 연동을 통한 웨이퍼 이미지 직접 업로드 지원 및 설계 도면 기반 이상 탐지 모델 통합
- **지능형 피드백 루프 구축**
LangSmith 평가 결과를 기반으로 RAG 청킹 전략 및 임베딩 모델을 실시간 최적화하는 자동 튜닝 파이프라인
- **설비 도메인 도구 생태계 확대**
온도, 압력, 유량 등 공정 전반의 센서 데이터를 해석하는 신규 진단 Tool 추가 및 에이전트 지식 고도화
- **Human-in-the-Loop 도입**
AI의 중요 판정 결과에 대해 현장 엔지니어의 피드백 및 승인 단계를 삽입하여 시스템 신뢰성 및 안전성 확보

결론 및 느낀점

✓ 프로젝트 완성도 및 성과

ML(week1) → DL(week2) → LLM(week3) 3세대 프로젝트의 통합

기존 학습·배포된 ML/DL 모델 산출물의 성공적인 Tool화 및 재사용

환경 독립성 확보 : 클라우드 API와 로컬 sLLM 간의 유연한 백엔드 전환

💡 핵심 기술적 인사이트

LLM의 본질은 해결사가 아닌 시스템을 연결하는 '오케스트레이터'

LangGraph를 통한 복잡한 분기 로직의 시각화 및 제어 가능성 확인

명확한 입출력 스키마(Tool 설계)가 전체 시스템 신뢰도의 핵심

INTEGRATED FACTORY INTELLIGENCE

3-Generation Evolution

ML: BEARING DIAGNOSTICS (week1)

DL: WAFER DEFECT DETECTION (week2)

LLM: AGENTIC ORCHESTRATION (week3)

“ 데이터 수집 → 예측 → 해석의 과정을 LLM 에이전트 인터페이스로 완성하다 ”